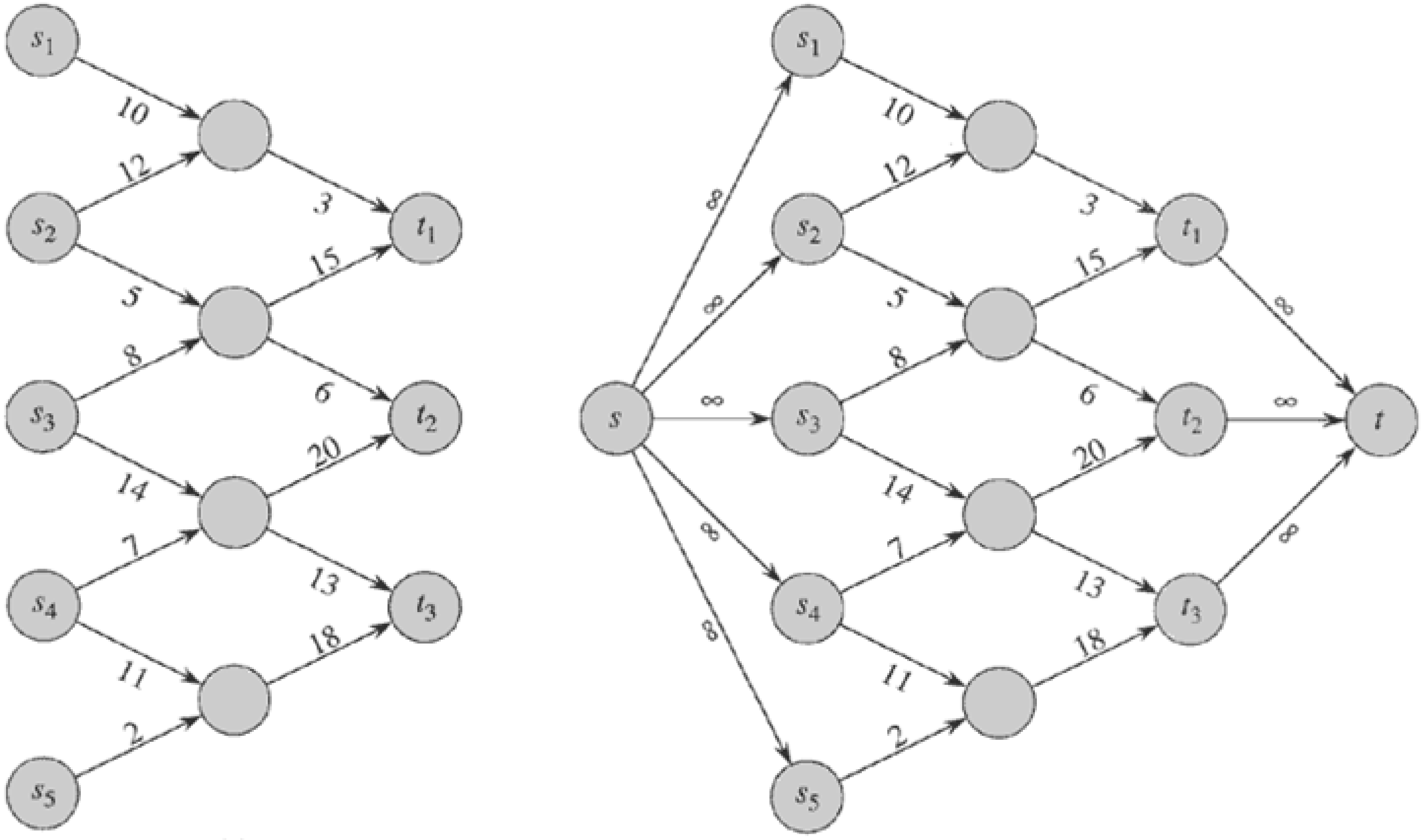
Edmonds-Karp Algorithm

- Proposed in 1972
- Almost same as Ford-Fulkerson
- Main difference: Uses BFS to find augmenting paths in residual graph instead of DFS
- You can prove that
  - If the Edmonds-Karp algorithm is run on a flow network $G = (V, E)$ with source s and sink t, then for all vertices $v \in V - \{s, t\}$, the shortest distance $\delta_f(s, v)$ in the residual network G_f increases monotonically with each flow augmentation
  - The total number of flow augmentations performed by the Edmonds-Karp algorithm is $O(VE)$
- This gives time complexity of Edmonds-Karp as $O(VE^2)$, as BFS can be done in $O(E)$

What if there are multiple sources and sink?

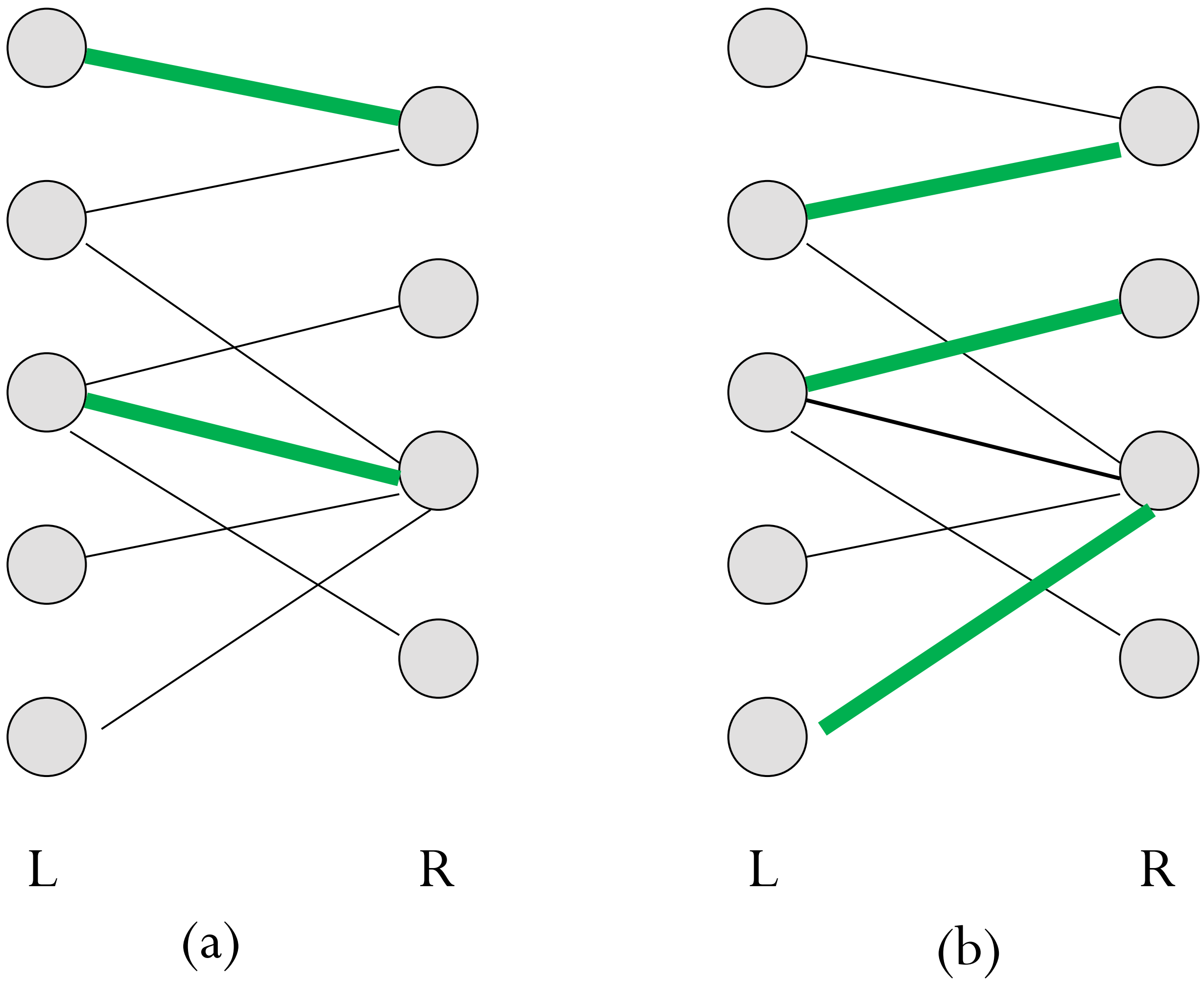
- Suppose there are multiple sources s_1, s_2, s_3,..s_p and multiple sinks t_1, t_2, t_3, ….t_q
- How do we maximize the sum of the flows from all the sources to all the sinks?
- Can easily use the standard maximum flow problem
  - Add a "supersource" s with edge (s, s_j) from s to all sources s_j with capacity ∞
  - Add a "supersink" t with edge (t_j, t) from all sinks t_j to t with capacity ∞
  - Solve the maximum flow problem with s as source and t as sink



(a) (b)

Application: Maximum Cardinality Bipartite Matching

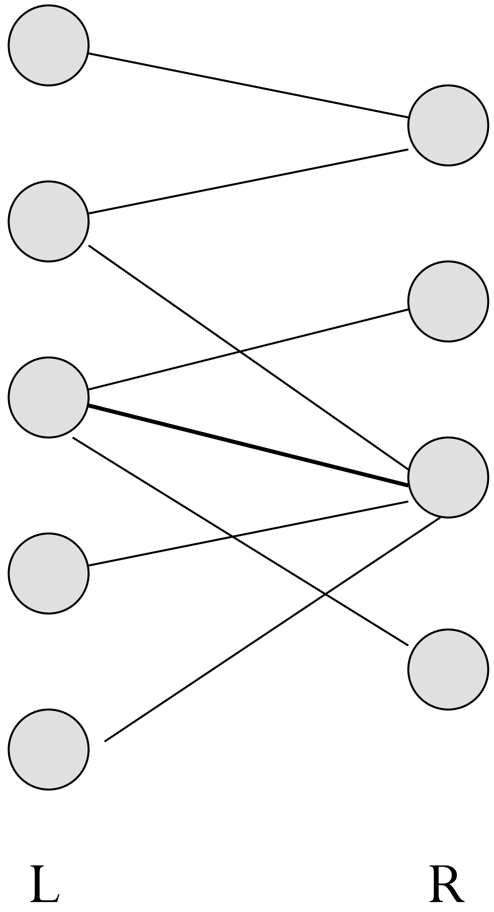
- Bipartite Graph: an undirected graph $G = (V, E)$ such that the vertex set can be partitioned $V = L \cup R$ where L and R are disjoint and there is no edge between two vertices in L or two vertices in R
- A matching in an undirected graph $G = (V, E)$ is a subset of edges $M \subseteq E$, such that for all vertices $v \in V$, at most one edge of M is incident on v.
- A maximum cardinality matching is a matching with maximum number of edges among all possible matchings
  - Also simply called maximum matching for unweighted graphs

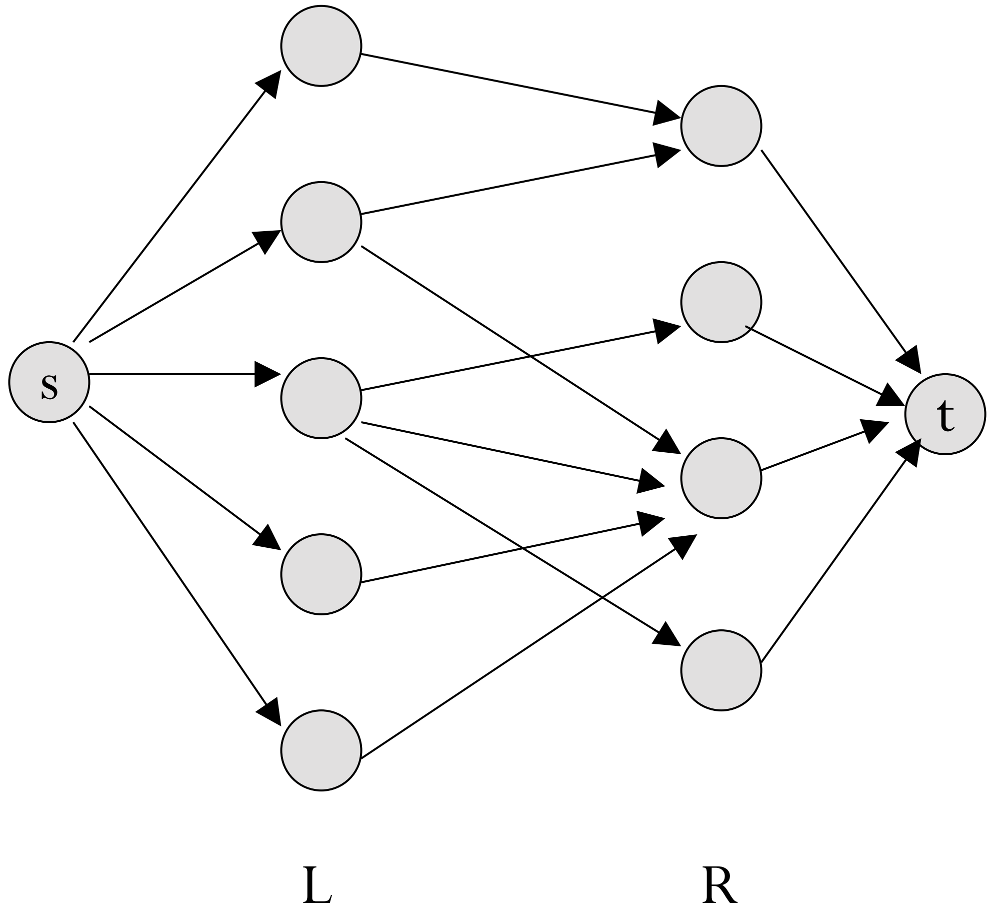


(a)A matching with cardinality 2

(b) A maximum matching with cardinality 3

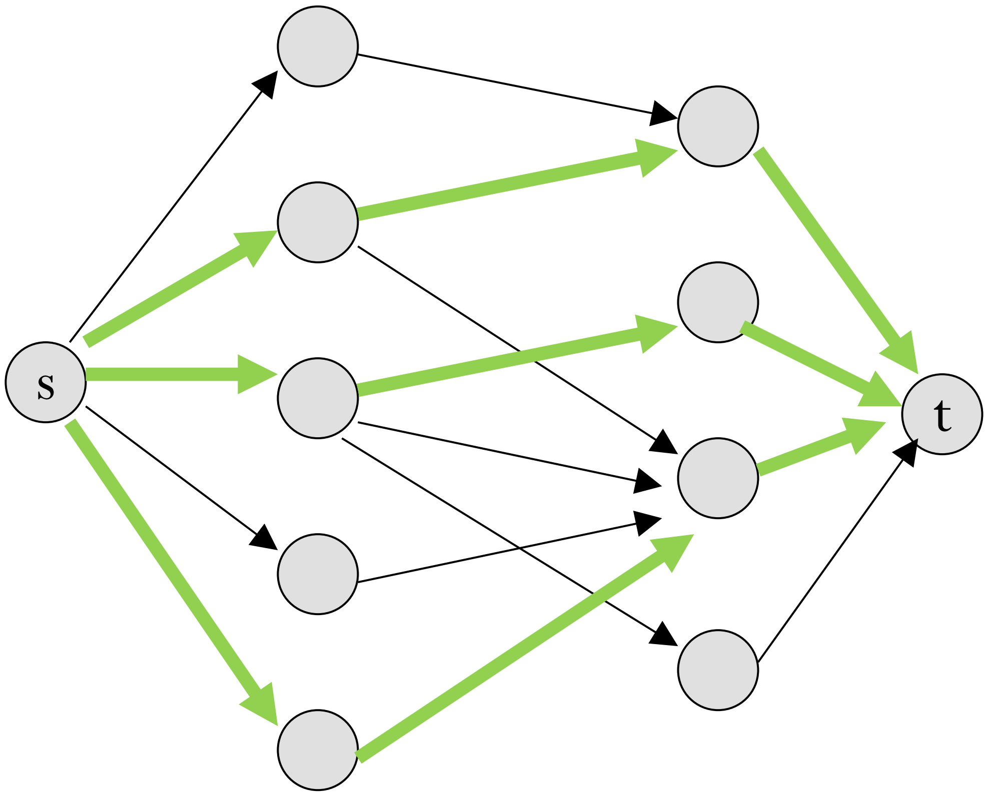
- Given the undirected bipartite graph G = (V, E) with partitions L and R, create a flow network G' = (V', E') as follows
  - Add two new vertices s, t. So $V' = V \cup \{s, t\}$
  - For each node u in L, add a directed edge (s,u) with capacity 1 to E'
  - For each node v in R, add a directed edge (v,t) with capacity 1 to E'
  - For each edge (u,v) in E with u in L and v in R, add a directed edge (u, v) with capacity 1 to E'



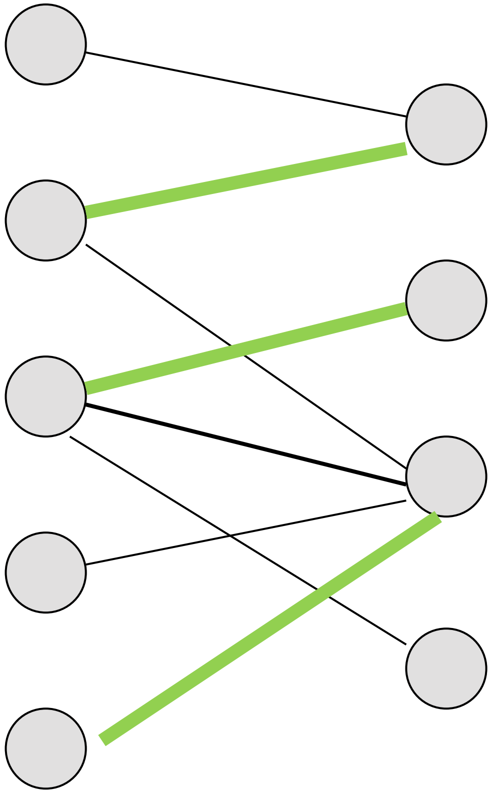


All capacities are 1

- Now solve the maximum flow problem from s to t in G'
- The edges of G with corresponding edges in G' with flow = 1 correspond to the maximum matching



Maximum flow found



Corresponding Maximum Matching